



Assignment 1

Released: 29 October 2025

Due: 10 November 2025 (via Canvas, before 23:59)

Group number:

Name, student id

Name, student id

Points, grade of 100 points **grade:**

Introduction

The objective of this assignment is to test your understanding of logical programming using Prolog. You are to work on this assignment in **groups of 2 students**.

Programming is something which cannot be learned without actually doing it, and the assignment will therefore ask you to complete several exercises on writing Prolog programs. This assignment covers Prolog databases and queries, matching, proof trees, and data types. It requires you to have a good understanding of the material covered in the lectures and the reference materials on [Learn Prolog Now](#), Chapters 1, 2, 4.1, 4.2, 5.1, 5.2, 10.3.

- **Working in pairs** does not mean to split the exercises and to solve them independently of your group member. We want you to work out the solutions *together*. Each student should understand all the exercises and be able to explain the solution. **This will be tested in the final exam.**
- You are required to have run and test Prolog code on your own machine, which is part of assignment 0, prior to working on this assignment. We use SWI Prolog version 9.2.9-1 for this course (latest stable at time of writing).
- Create one .pl file and include your Prolog solutions and written explanation and motivation as comments. Submit the .pl file on Canvas. Note: only one member of the group needs to submit it.

Exercises

Exercise 1 (Representation in Prolog - 21 pts).

Specify the following in Prolog in a functional knowledge base:

1. *The Lord of the Rings* is a fantasy novel. [2 pts]
2. *The Lord of the Rings* is a fantasy novel and a film trilogy. [2 pts]
3. *The Lord of the Rings* is written by J.R.R. Tolkien, and Tolkien was a fantasy writer. [2 pts]
4. A being who possesses the One Ring becomes corrupted unless they are of pure heart. [5 pts] (Write a rule with head `corruptionCheck/2`, returning true if the being remains uncorrupted and false otherwise.)
5. A being who resists corruption may carry the Ring safely only if they do not use its power. [5 pts] (Write a rule with head `safeBearer/2`, returning true if the being can safely bear the Ring and false otherwise; assume the rule is appended to the knowledge base that already includes `corruptionCheck/2`.)
6. A being may destroy the Ring at mount Doom only if they are a safe bearer. [5 pts] (Write a rule with head `canDestroy/2`, returning true if the being can destroy the Ring and false otherwise; assume the rule is appended to the knowledge base that already includes the other two rules.)

Exercise 2 (Ordering and negation - 12 pts).

Extend the knowledge base created in the previous exercise with facts accompanying the Lord of the Ring rules (i.e. the rules you written for 4 - 6 of Exercise 1). You are free to choose facts and the atoms of your choice, as long as the predicates could be matched. Include your queries and their results as a comment.

1. The knowledge base should make all three rules queryable (e.g. a query of 'safeBearer/2' returns the entity that qualifies as a safeBearer of the ring). [6 pts]
2. The knowledge base should also allow a query that returns false for each rule. [6 pts]

Exercise 3 (Unification - 18 pts).

For the following prolog clauses that pair terms, indicate which of the pairs unify and, if they unify, give the variable instantiations that lead to successful unification.

1. `ring = power`. [1.5 pts]
2. `power = Ring`. [1.5 pts]
3. `key(X,'value') = key(data,X)`. [1.5 pts]
4. `triple(X,item(Y),Z) = triple(a,item(b),c)`. [1.5 pts]
5. `structure(X,layer(Y),depth(Z)) = structure(layer(a),layer(b),depth(c))`. [1.5 pts]
6. `X = 3 + 4`. [1.5 pts]
7. `Y is 3 + 4`. [1.5 pts]
8. `7 ::= 3 + 4`. [1.5 pts]

9. $7 \neq 2 * 3$. [1.5 pts]
10. Z is $5 * 2$, $Z := 10$. [1.5 pts]
11. $\neg (5 = 6)$. [1.5 pts]
12. C is $(7 - 2)$, $C := 5$. [1.5 pts]

Exercise 4 (Ordering and proof trees - 21 pts).

Consider the following knowledge base:

```
mammal(dog).
mammal(cat).
bird(eagle).
bird(parrot).
```

```
same_species(X,Y) :- not(Y = X), mammal(X), mammal(Y).
same_species(X,Y) :- not(Y = X), bird(X), bird(Y).
```

1. The knowledge base is not functional in this set-up; a call like *same_species(dog, A)*. will fail. Explain why this is the case. [5 pts]
2. Adapt the knowledge base in such a way that the call will succeed. Specify the crucial lines and explain why the call given in Exercise 4.1 will now give the appropriate answer. [5 pts]
3. Draw the search tree for the call *same_species(dog, A)*. (from the adapted knowledge base after completing Exercise 2.2). For drawing the tree, an ASCII representation in Prolog code is acceptable. [6 pts]
4. The current search order within the rules may still cause unnecessary backtracking, especially when variables are uninstantiated. Reorder the goals *within each rule* (not the rules themselves) to make the program more efficient, and explain why this change improves performance. [5 pts]

Exercise 5 (String and List Manipulation - 16 pts).

Write Prolog facts and rules that succeed through simple pattern matching. All goals must be provable through unification alone.

1. Define a predicate *first_two(List, X, Y)* that succeeds when X and Y are the first two elements of List. Example query: *?- first_two([a,b,c,d], X, Y)*. [4 pts]
2. Define a predicate *last_two(List, X, Y)* that succeeds when X and Y are the last two elements of a list of exactly four elements. Example query: *?- last_two([a,b,c,d], X, Y)*. [4 pts]
3. Define a predicate *split_string(StringList, Prefix, Suffix)* that succeeds when StringList is composed of two sublists, the first of length two (Prefix) and the rest (Suffix). Example query: *?- split_string([h,e,l,l,o], P, S)*. [4 pts]
4. Define a predicate *swap_pair(Pair, Swapped)* that succeeds when Pair is a list of exactly two elements and Swapped is that list with the order reversed. Example query: *?- swap_pair([x,y], Z)*. [4 pts]

Constraints:

- Do *not* use recursion.
- Do *not* use any predefined list or string predicates.
- Solutions must rely solely on pattern matching and unification.

Exercise 6 (ordering - 12 pts).

Consider the following Prolog clauses:

```
summer(X):-happy(X).  summer(X):-warm(Y).  warm(Y):-warm(Y).  warm(a).  happy(b).
```

Let the rules be enumerated from 1 to 5 (from left to right). Use all these clauses to construct Prolog programs that only differ in the order of the clauses. For each of the following cases construct a program P such that on query `?-summer(a)`

1. ...directly runs into an infinite loop without returning any answer. [6 pts]
2. ...returns infinitely often the answer 'true'. [6 pts]

Give the order of the rules and explain why these results occur.